

NAME- Gunjan Chute

PRN NO- 202501050025

BATCH- CH2

PRACTICAL NO.4

4.1.1. Pandas - series creation and manipulation

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the groupby and mean() operations.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

CODE:

```
import pandas as pd
```

```
# Take inputs from the user to create a list of numbers numbers  
= list(map(int, input().split()))
```

```
# Create a Pandas series from the list of numbers series
```

```
= pd.Series(numbers)

# Grouping by even and odd numbers and calculating the mean grouped =
series.groupby(series%2==0).mean()

# Display the mean of even and odd numbers with labels
grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
print("Mean of even and odd numbers:") print(grouped)
```

OUTPUT:

4.1.2. Dictionary to dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).

- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the **Age** column.
- Display the DataFrame after deleting the column.

CODE:

```
import pandas as pd
```

```
# Provided dictionary of lists data
```

```
= {
```

```
    'Name': ['Alice', 'Bob', 'Charlie'],
```

```
    'Age': [25, 30, 35],
```

```
}
```

```
# Convert the dictionary to a DataFrame df
```

```
= pd.DataFrame(data)
```

```
# Display the original DataFrame print("Original
```

```
DataFrame:") print(df)
```

```
# Adding a new row new_name = input("New name:
```

```
") new_age = int(input("New age: ")) new_row =
```

```
{'Name': new_name, 'Age': new_age } df =
```

```
df.append(new_row,ignore_index = True) # Display
```

```
the DataFrame after adding a new row print("After
```

```
adding a row:\n",df)
```

```
# Modifying a row
```

```
row_to_modify = int(input("Index of row to modify: ")) new_age_value
```

```
= int(input("New age: ")) df.at[row_to_modify, "Age"] =
```

```
new_age_value # Display the DataFrame after modifying a row
```

```
print("After modifying a row:") print(df) row_to_delete =
```

```
int(input("Index of row to delete: ")) df =
```

```
df.drop(index=row_to_delete).reset_index(drop=True)
```

```
# Deleting a row
```

```
# Display the DataFrame after deleting a row print("After deleting
```

```
a row:") print(df)
```

Adding a new column

```
genders = input("Enter genders separated by space: ").split() df["Gender"] =  
genders
```

Display the DataFrame after adding a new column print("After adding
a new column:") print(df)

Modifying a column df['Name'] = df['Name'].str.upper()

Display the DataFrame after modifying a column print("After
modifying a column:") print(df)

Deleting a column

```
df = df.drop(columns='Age').reset_index(drop=True) #
```

Display the DataFrame after deleting a column print("After
deleting a column:") print(df)

OUTPUT:

The screenshot shows a Jupyter Notebook interface with a dark theme. The top bar includes the logo 'C@DE TANTRA' and user information '202010190021@nitk.ac.in'. The notebook displays a series of code cells and their outputs. The code performs the following operations:

- Adding a new column 'Gender' to a DataFrame 'df'.
- Displaying the DataFrame after adding the new column.
- Modifying the 'Name' column to uppercase.
- Displaying the DataFrame after modifying the column.
- Deleting the 'Age' column.
- Displaying the DataFrame after deleting the column.

The outputs show the state of the DataFrame at each step, with row indices and column names clearly visible. The final output shows a DataFrame with columns 'Name' and 'Gender'.

4.1.3. Student Information

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

CODE:

```
import pandas as pd
```

```
# Read the text file into a DataFrame file
```

```
= input() data = pd.read_csv(file, sep="\s+", header=None, names=["Name",  
"Age", "Grade"])
```

```
# write your code here.. print("First five rows:")
```

```
print(data.head())
```

```
average_age= round(data["Age"].mean(), 2) print(f"Average age:
```

```
{average_age}") filtered_student = data[data["Grade"] <= "B"]
```

```
print("Students with a grade up to B")
```

```
print(filtered_student)
```

OUTPUT:

[Home](#)
[Learn Anywhere](#)

202501050025@mitaoe.ac.in
Support
Logout

Average time
0.082 s
82.00 ms

Maximum time
0.120 s
120.00 ms

1 out of 1 shown test case(s) passed
1 out of 1 hidden test case(s) passed

Test case 1
120 ms
Debug

Expected output

Actual output

```

studentdata.txt
First-five rows:
...Name...Age...Grade...
0..John...25...A...
1..Allin...22...B...
2..Emma...24...A...
3..Mike...23...C...
4..Sony...21...B...
Average age: 23.29
Students with a grade up to B:
...Name...Age...Grade...
0..John...25...A...
1..Allin...22...B...
2..Emma...24...A...
4..Sony...21...B...
5..Amia...26...A...

```

```

studentdata.txt
First-five rows:
...Name...Age...Grade...
0..John...25...A...
1..Allin...22...B...
2..Emma...24...A...
3..Mike...23...C...
4..Sony...21...B...
Average age: 23.29
Students with a grade up to B:
...Name...Age...Grade...
0..John...25...A...
1..Allin...22...B...
2..Emma...24...A...
4..Sony...21...B...
5..Amia...26...A...

```

< Prev
Reset
Submit
Next >

4.2.1. Month with the Highest Total Sales

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

• Sample Data:

- Date,Product,Quantity,Price,City
- 2025-01-01,Product A,5,20,New York
- 2025-01-01,Product B,3,15,Los Angeles
- 2025-01-02,Product A,7,20,New York
- 2025-01-02,Product C,4,30,Chicago
- 2025-01-03,Product B,2,15,Chicago
- 2025-01-03,Product A,8,20,Los Angeles
- 2025-01-04,Product C,6,30,New York
- 2025-01-04,Product B,5,15,Los Angeles
- 2025-01-05,Product A,3,20,Chicago
- 2025-01-05,Product C,10,30,Los Angeles

CODE:

```
import pandas as pd

# Prompt the user for the file name file_name
file_name = input()

# Load the data df =
df = pd.read_csv(file_name)

df["Date"] = pd.to_datetime(df["Date"])

df["Month"] = df["Date"].dt.to_period("M")

df["Total Sales"] = df["Quantity"] * df["Price"]

monthly_sales = df.groupby("Month")["Total Sales"].sum()

best_month = monthly_sales.idxmax()
highest_sales = monthly_sales.max()
print(f"Best month: {best_month}")
print(f"Total sales: ${highest_sales:.2f}")
```

OUTPUT:

[Home](#)
[Learn Anywhere](#)

202501050025@mitaoe.ac.in
[Support](#)
[Logout](#)

Average time

0.057 s

57.00 ms

Maximum time

0.109 s

109.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

109 ms

Debug

Expected output

Actual output

sales_data.csv

sales_data.csv

Best month: 2025-01

Best month: 2025-01

Total sales: \$1210.00

Total sales: \$1210.00

< Prev

Reset

Submit

Next >

4.2.2. Best Selling Product

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:
Date,Product,Quantity,Price,City 2025-01-01,Product A,5,20,New York 2025-01-01,Product B,3,15,Los Angeles 2025-01-02,Product A,7,20,New York 2025-01-02,Product C,4,30,Chicago 2025-01-03,Product B,2,15,Chicago 2025-01-03,Product A,8,20,Los Angeles 2025-01-04,Product C,6,30,New York 2025-01-04,Product B,5,15,Los Angeles 2025-01-05,Product A,3,20,Chicago 2025-01-05,Product C,10,30,Los Angeles

CODE:

```
import pandas as pd
```

```
# Prompt the user for the file name file_name
```

```
= input()
```

```
# Load the data df =
```

```
pd.read_csv(file_name) product_sales =
```

```
df.groupby("Product")["Quantity"].sum()
```

```
best_product = product_sales.idxmax() highest_quantity
```

```
= product_sales.max() # Display the result
```

```
print(f"Best selling product: {best_product}") print(f"Total
```

```
quantity sold: {highest_quantity}")
```

OUTPUT:

The screenshot displays the CodeTANTRA IDE interface. At the top, the header includes the CodeTANTRA logo, navigation links (Home, Learn Anywhere), a user profile (202501050025@mitaoe.ac.in), and a Logout button. The main workspace is divided into several sections. On the left, there is an Explorer panel. The central area shows the execution results for a test case. It includes a table with 'Average time' (0.038 s) and 'Maximum time' (0.081 s). Below this, a green banner indicates '1 out of 1 shown test case(s) passed' and '2 out of 2 hidden test case(s) passed'. A 'Test case 1' section shows the 'Expected output' and 'Actual output' for the file 'sales_data.csv'. The output for both is identical: 'Best-selling product: Product-A' and 'Total quantity sold: 23'. At the bottom right, there are buttons for '< Prev', 'Reset', 'Submit', and 'Next >'. A 'Debugger' panel is visible on the far right.

Metric	Value
Average time	0.038 s
Maximum time	0.081 s

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 (81 ms) [Debug]

Expected output	Actual output
sales_data.csv	sales_data.csv
Best-selling product: Product-A	Best-selling product: Product-A
Total quantity sold: 23	Total quantity sold: 23

< Prev Reset Submit Next >

4.2.3. City that Sold the Most Products

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

• Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

CODE:

```
import pandas as pd
```

```
# Prompt the user for the file name file_name
```

```
= input()
```

```
# Load the data df =
```

```
pd.read_csv(file_name)
```

```
# write the code.. city_sales =
df.groupby("City")["Quantity"].sum()
best_city = city_sales.idxmax()
# Display the result
print(f"City sold the most products: {best_city}")
```

OUTPUT:

The screenshot displays the CODETANTRA IDE interface. At the top, the header includes the CODETANTRA logo, navigation links for Home and Learn Anywhere, a user profile with email 202501050025@mitaoe.ac.in, a Support link, and a Logout button. The main workspace is divided into several sections. On the left, an Explorer sidebar shows a file named sales_data.csv. The central area displays the test results for 'Test case 1', which passed with a duration of 102 ms. Above the test results, a summary box indicates that 1 out of 1 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. The test case details show the expected output as 'City sold the most products: Los Angeles' and the actual output as 'City sold the most products: Los Angeles', both matching. The bottom of the interface features a dark blue bar with navigation buttons: < Prev, Reset, Submit, and Next >.

4.2.4. Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

•

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

CODE:

```
import pandas as pd

from itertools import combinations from collections

import Counter

# Prompt user to input the file name file_name
file_name = input()

# Read data from the specified CSV file df
df = pd.read_csv(file_name)

# write the code
grouped = df.groupby("Date")["Product"].apply(list)
```

Output the most frequent product pairs

pairs_counter = Counter() for products in

grouped:

pairs = combinations(sorted(products), 2)

pairs_counter.update(pairs) max_frequency = max(pairs_counter.values())

most_common_pairs = [(pair, freq) for pair, freq in pairs_counter.items() if freq == max_frequency]

for (product1,product2),frequency in most_common_pairs: print(f"{product1} and {product2}: {frequency} times")

OUTPUT:

The screenshot displays the CodeTANTRA IDE interface. At the top, the header includes the CodeTANTRA logo, navigation links for Home and Learn Anywhere, a user profile with email 202501050025@mitaoe.ac.in, a Support link, and a Logout button. The main workspace is divided into several sections. On the left, an Explorer panel shows a file named sales_data.csv. The top center area displays performance metrics: Average time (0.042 s, 41.67 ms) and Maximum time (0.083 s, 83.00 ms). To the right of these metrics, a green banner indicates '1 out of 1 shown test case(s) passed' and '2 out of 2 hidden test case(s) passed'. Below this, a 'Test case 1' section shows a green checkmark, a green '83 ms' duration, and a 'Debug' button. The 'Expected output' section lists 'sales_data.csv', 'Product A and Product B: 2 times', and 'Product A and Product C: 2 times'. The 'Actual output' section shows the same results. At the bottom, a 'Terminal' and 'Test cases' tab are visible. The bottom right corner features navigation buttons: '< Prev', 'Reset', 'Submit', and 'Next >'.

4.2.5. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

P as se ng er id	Su rvi ve d	Pc la ss	N a m e	S e x	Ag e	Si bS p	P ar ch	Ti ck et	Fa re	C ab in	E m ba rk ed

CODE:

```
import pandas as pd
import numpy
as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# 1. Display the first 5 rows of the dataset
print(data.head())

# 2. Display the last 5 rows of the dataset
print(data.tail())

# 3. Get the shape of the dataset
print(data.shape)

# 4. Get a summary of the dataset (info)
print(data.info())

# 5. Get basic statistics of the dataset
print(data.describe())

# 6. Check for missing values
print(data.isnull().sum())

# 7. Fill missing values in the 'Age' column with the median age
median_age = data["Age"].median()
data["Age"].fillna(median_age, inplace=True)

# 8. Fill missing values in the 'Embarked' column with the mode
mode_embarked = data['Embarked'].mode()[0]

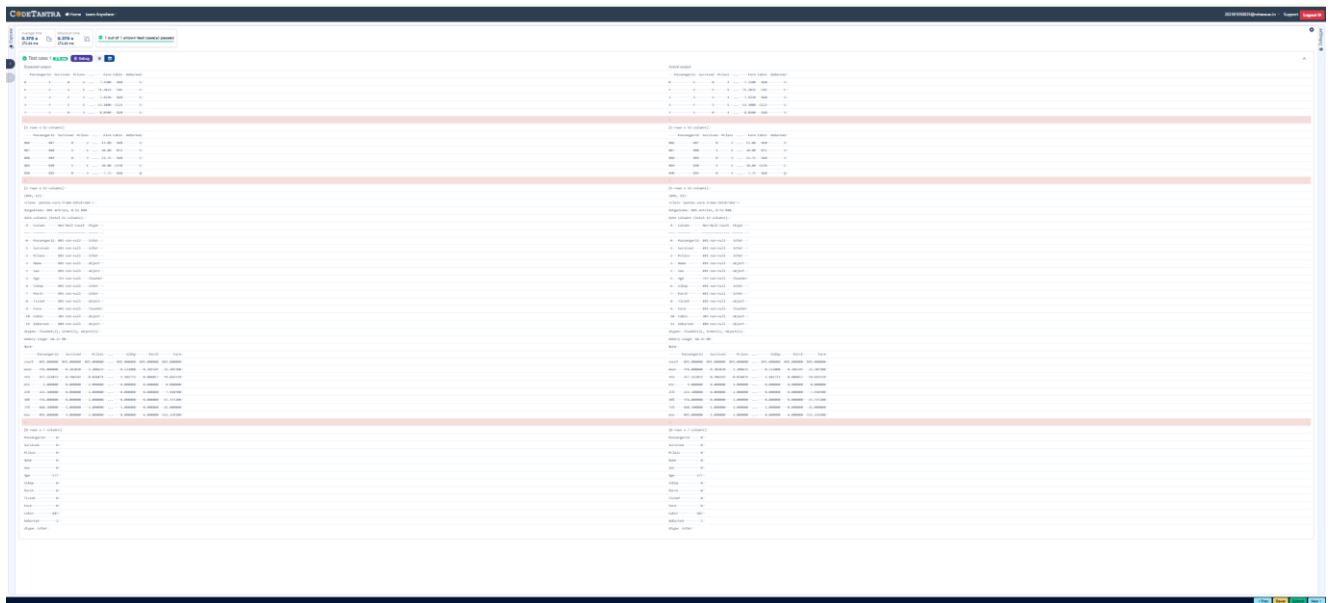
data['Embarked'].fillna(mode_embarked, inplace=True)

# 9. Drop the 'Cabin' column due to many missing values
data.drop('Cabin', axis=1, inplace=True)
```


10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'

```
data['FamilySize'] = data['SibSp'] + data['Parch']
```

OUTPUT:



The screenshot shows a Jupyter Notebook interface with a code cell and its output. The code cell contains the following Python code:

```
data['FamilySize'] = data['SibSp'] + data['Parch']
```

The output displays a table with 12 columns: PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, FamilySize, Fare, Cabin, and Embarked. The 'FamilySize' column is the result of adding 'SibSp' and 'Parch' for each passenger. The table is divided into three sections by horizontal lines.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	FamilySize	Fare	Cabin	Embarked
1	0	3	Mr. Braund	male	22	0	0	0	5.42	C51	S
2	1	1	Mr. Allen	male	35	1	0	1	53.1	B5	C
3	0	3	Mr. Heikkinen	male	26	0	0	0	5.18	NaN	S
4	1	1	Mr. Murray	male	30	0	0	0	51.66	B5	C
5	0	3	Mr. Lehto	male	19	0	0	0	5.18	NaN	S
6	0	3	Mr. Maki	male	19	0	0	0	5.18	NaN	S
7	0	3	Mr. Maki	male	19	0	0	0	5.18	NaN	S
8	0	3	Mr. Maki	male	19	0	0	0	5.18	NaN	S
9	0	3	Mr. Maki	male	19	0	0	0	5.18	NaN	S
10	0	3	Mr. Maki	male	19	0	0	0	5.18	NaN	S

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.

9. Calculate the survival rate of passengers.

10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below

P as se ng er id	Su rvi ve d	Pc la ss	N a m e	S ex	Ag e	Si bS p	P ar ch	Ti ck et	Fa re	C ab in	E m ba rk ed

CODE:

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv') data['FamilySize'] = data['SibSp']  
+ data['Parch']
```

```
# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise) data['IsAlone']  
= np.where(data['FamilySize'] == 0, 1, 0)
```

```
# 2. Convert 'Sex' to numeric (male: 0, female: 1) data['Sex']  
= data['Sex'].map({'male': 0, 'female': 1})
```

```
# 3. One-hot encode the 'Embarked' column data = pd.get_dummies(data,  
columns=['Embarked'])
```

```
# 4. Get the mean age of passengers  
mean_age = data['Age'].mean()  
print( mean_age)
```

5. Get the median fare of passengers

```
median_fare = data['Fare'].median() print(  
median_fare)
```

6. Get the number of passengers by class print(data['Pclass'].value_counts())

7. Get the number of passengers by gender print(data['Sex'].value_counts())

8. Get the number of passengers by survival status print(
data['Survived'].value_counts())

9. Calculate the overall survival rate survival_rate =

```
data['Survived'].mean() print(format(survival_rate)) # 10. Calculate the  
survival rate by gender
```

```
survival_by_gender = data.groupby('Sex')['Survived'].mean() print(  
survival_by_gender)
```

OUTPUT:

[Home](#)
[Learn Anywhere](#)

202501050025@mitaoe.ac.in
Support
Logout

Average time
0.197 s
197.00 ms

Maximum time
0.197 s
197.00 ms

1 out of 1 shown test case(s) passed

Test case 1
197 ms
Debug

Expected output	Actual output
29.69911764705882	29.69911764705882
14.4542	14.4542
3...491	3...491
1...216	1...216
2...184	2...184
Name: Pclass, dtype: int64	Name: Pclass, dtype: int64
0...577	0...577
1...314	1...314
Name: Sex, dtype: int64	Name: Sex, dtype: int64
0...549	0...549
1...342	1...342
Name: Survived, dtype: int64	Name: Survived, dtype: int64
0.3838383838383838	0.3838383838383838
Sex	Sex
0...0.188908	0...0.188908
1...0.742038	1...0.742038
Name: Survived, dtype: float64	Name: Survived, dtype: float64

Prev
Reset
Submit
Next

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

P as se ng er id	Su rvi ve d	Pc la ss	N a m e	S ex	Ag e	Si bS p	P ar ch	Ti ck et	Fa re	C ab in	E m ba rk ed

CODE:

```
import pandas as pd
import numpy
```

```
as np
```

```
# Load the Titanic dataset data = pd.read_csv('Titanic-Dataset.csv')
```

```
data['FamilySize'] = data['SibSp'] + data['Parch'] data['IsAlone'] =
```

```
np.where(data['FamilySize'] > 0, 0, 1) data = pd.get_dummies(data,
columns=['Embarked'], drop_first=True)
```

```
# 1. Calculate the survival rate by class
```

```
print(data.groupby('Pclass')['Survived'].mean())
```

```
# 2. Calculate the survival rate by embarked location
```

```
print(data.groupby('Embarked_S')['Survived'].mean()) # 3. Calculate the
```

```
survival rate by family size print(data.groupby('FamilySize')['Survived'].mean())
```

```
# 4. Calculate the survival rate by being alone
```

```
print(data.groupby('IsAlone')['Survived'].mean())
```

```
# 5. Get the average fare by class print(data.groupby('Pclass')['Fare'].mean())
```

```
# 6. Get the average age by class print(data.groupby('Pclass')['Age'].mean())
```

```
# 7. Get the average age by survival status
```

```
print(data.groupby('Survived')['Age'].mean())
```

```
# 8. Get the average fare by survival status
```

```
print(data.groupby('Survived')['Fare'].mean())
```

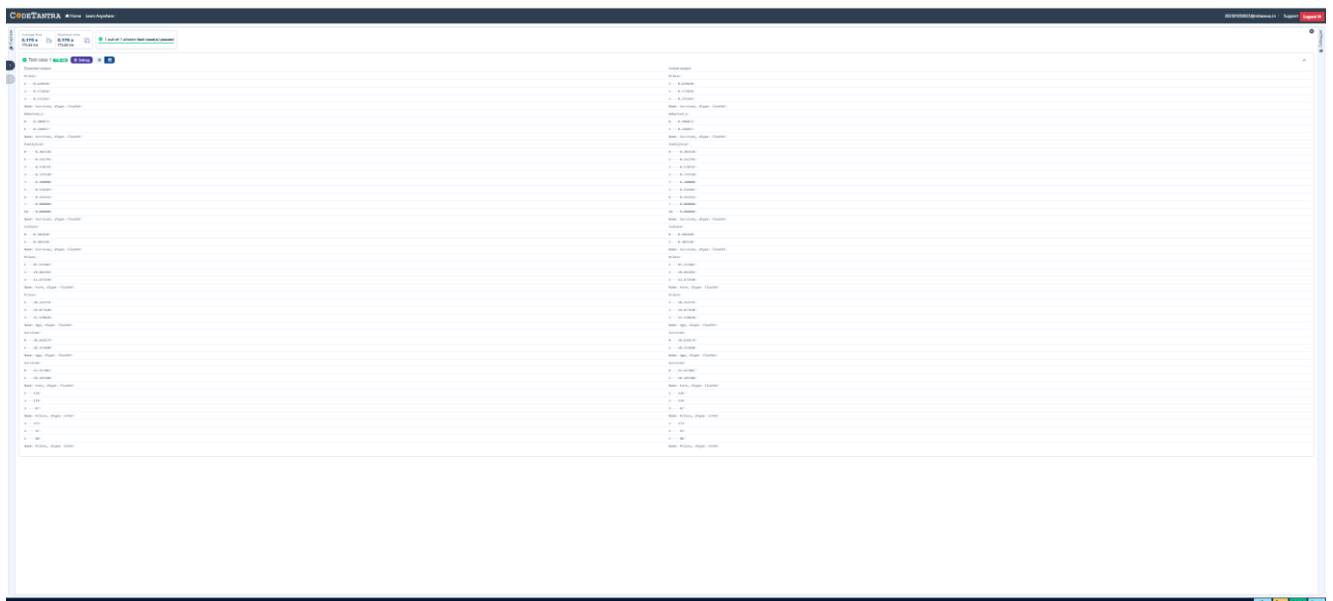
```
# 9. Get the number of survivors by class
```

```
print(data[data['Survived'] == 1] ['Pclass'].value_counts())
```

```
# 10. Get the number of non-survivors by class print(data[data['Survived'] == 0]
```

```
['Pclass'].value_counts())
```

OUTPUT:



4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

P as se ng er d	Su rvi ve d	Pc la ss	N a m e	S ex	Ag e	Si bS p	P ar ch	Ti ck et	Fa re	C ab in	E m ba rk ed

CODE:

```
import pandas as pd
import numpy as np
```

```

# Load the Titanic dataset data = pd.read_csv('Titanic-Dataset.csv') data
= pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 1. Get the number of survivors by gender print(data[data['Survived'] ==
1]['Sex'].value_counts())

# 2. Get the number of non-survivors by gender print(data[data['Survived'] ==
0]['Sex'].value_counts())

# 3. Get the number of survivors by embarked location
print(data[data['Survived'] == 1]['Embarked_S'].value_counts()) # 4. Get the
number of non-survivors by embarked location print(data[data['Survived'] ==
0]['Embarked_S'].value_counts())

# 5. Calculate the percentage of children (Age < 18) who survived
children = data[data['Age'] < 18] print(children['Survived'].mean()) #
6. Calculate the percentage of adults (Age >= 18) who survived adults
= data[data['Age'] >= 18] print(adults['Survived'].mean())

# 7. Get the median age of survivors print(data[data['Survived']
== 1]['Age'].median())

# 8. Get the median age of non-survivors print(data[data['Survived']
== 0]['Age'].median())

# 9. Get the median fare of survivors print(data[data['Survived']
== 1]['Fare'].median())

```



```
# 10. Get the median fare of non-survivors print(data[data['Survived'] ==  
0]['Fare'].median())
```

OUTPUT:

CODETANTRA Home Learn Anywhere 202501050025@mitaoe.ac.in Support Logout

Average time: 0.192 s, 192.00 ms Maximum time: 0.192 s, 192.00 ms 1 out of 1 shown test case(s) passed

Test case 1 192 ms Debug

Expected output	Actual output
female... 233	female... 233
male... 109	male... 109
Name: Sex, dtype: int64	Name: Sex, dtype: int64
male... 468	male... 468
female... 81	female... 81
Name: Sex, dtype: int64	Name: Sex, dtype: int64
1... 217	1... 217
0... 125	0... 125
Name: Embarked_S, dtype: int64	Name: Embarked_S, dtype: int64
1... 427	1... 427
0... 122	0... 122
Name: Embarked_S, dtype: int64	Name: Embarked_S, dtype: int64
0.5398230088495575	0.5398230088495575
0.3810316139767055	0.3810316139767055
28.0	28.0
28.0	28.0
26.0	26.0
10.5	10.5

Prev Reset Submit Next